

TNY — Documentação Técnica (Detalhada)

TNY Catálogo é um sistema fullstack para catálogo digital e gestão de pedidos de uma marca de moda masculina. Clientes navegam pelos produtos, montam o carrinho e finalizam o pedido via WhatsApp. Administradores gerenciam o catálogo, pedidos e leads por meio de um painel protegido por JWT.

Observação de propósito: este documento descreve não só como usar e rodar o projeto, mas também as escolhas arquiteturais e de tecnologia que orientaram sua implementação. Onde aplicável, explicamos os trade-offs e motivações para facilitar manutenção e futuras evoluções.

Sumário

- 1. Visão Geral do Projeto
 - 2. Tecnologias e Dependências
 - 3. Pré-requisitos
 - 4. Instalação e Configuração
 - 5. Execução
 - 6. Estrutura do Projeto
 - 7. Arquitetura do Backend
 - 8. Banco de Dados
 - 9. API — Referência de Rotas
 - 10. Testes
 - 11. Containerização e Deploy
 - 12. Scripts — Referência Rápida
-

1. Visão Geral do Projeto

O que é o TNY?

O TNY é um sistema de catálogo de e-commerce voltado para uma marca de roupas masculinas. O sistema possui duas frentes:

- **Vitrine pública** — permite navegação por categorias e produtos, visualização de variantes (cor/tamanho), adição ao carrinho e finalização do pedido via WhatsApp.
- **Painel administrativo** — painel protegido para administradores gerenciarem catálogo, pedidos, leads e administradores.

Por que essa separação?

- A distinção entre vitrine e painel admin simplifica responsabilidades: a vitrine foca experiência do cliente (performance e usabilidade), enquanto o painel foca ferramentas administrativas e segurança (autenticação, auditoria).

Objetivo do Projeto

O TNY foi projetado para ser leve, de fácil implantação e escalável. Optamos por delegar a finalização do pedido ao WhatsApp para reduzir complexidade (pagamentos, conciliação, compliance) e acelerar o go-to-market. Esta decisão também facilita comunicação direta entre cliente e vendedor, importante para marcas de nicho que valorizam atendimento personalizado.

Trade-offs desta abordagem:

- Prós: simplicidade, menor custo de desenvolvimento, maior transparência no atendimento.
- Contras: ausência de pagamento online nativo e menor automação de processo de vendas (p. ex. confirmação automática de pagamento).

Funcionalidades Principais

As funcionalidades listadas abaixo priorizam clareza de escopo e separação de preocupações entre experiência cliente e operação administrativa.

Área	Funcionalidade
Vitrine	Listagem de produtos com filtro por categoria e busca textual
Vitrine	Página de detalhe do produto com galeria e seleção de variante
Vitrine	Produtos relacionados
Vitrine	Carrinho de compras persistido (localStorage)
Vitrine	Checkout com geração de link WhatsApp (mensagem pré-formatada)
Vitrine	Cadastro de lead (newsletter)
Admin	Autenticação JWT (login/logout/me)
Admin	CRUD de categorias, produtos, variantes e imagens
Admin	Gestão de pedidos (listagem + atualização de status)
Admin	Gestão de leads (listagem + exclusão)
Admin	Gestão de administradores
Sistema	Swagger UI com documentação interativa da API
Sistema	Health check endpoint

Cada funcionalidade foi desenhada para ser testável isoladamente e para permitir substituição de componentes (p. ex. trocar armazenamento local do carrinho por backend) sem grandes mudanças na arquitetura.

2. Requisitos do Sistema

Os requisitos representam as funcionalidades e as características de qualidade que o sistema deve possuir para atender às necessidades da empresa TNY e de seus usuários. Eles foram divididos em **Requisitos Funcionais**, que descrevem as funcionalidades esperadas, e **Requisitos Não Funcionais**, que definem padrões de desempenho, segurança, usabilidade e disponibilidade.

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem todas as ações que o sistema deve ser capaz de executar, permitindo que clientes e administradores realizem suas atividades de maneira eficiente.

RF01 – Visualizar Categorias de Produtos

O sistema deve permitir que os usuários consultem todas as categorias cadastradas, organizando os produtos de forma lógica e intuitiva.

Essa funcionalidade facilita a navegação do catálogo, melhora a experiência do usuário durante a busca por produtos e permite uma organização eficiente do estoque. Além disso, a categorização reduz o tempo necessário para localizar itens específicos e contribui para uma melhor apresentação dos produtos.

RF02 – Buscar Produtos

O sistema deverá disponibilizar uma ferramenta de pesquisa que permita localizar produtos utilizando diferentes filtros, como:

- Nome;
- Categoria;
- Código;
- Palavra-chave.

A busca rápida melhora significativamente a experiência do usuário, reduzindo o tempo necessário para encontrar um produto desejado. Essa funcionalidade também auxilia administradores durante a gestão do catálogo.

RF03 – Visualizar Produtos

O sistema deverá listar todos os produtos cadastrados apresentando informações relevantes, tais como:

- Nome;
- Imagem;
- Preço;
- Categoria;
- Descrição;
- Quantidade em estoque.

Essa funcionalidade fornece ao cliente todas as informações necessárias para tomar uma decisão de compra e permite que o administrador acompanhe facilmente os itens disponíveis.

RF04 – Adicionar Produtos ao Carrinho

O usuário poderá selecionar produtos e adicioná-los ao carrinho de compras.

O carrinho armazenará temporariamente os produtos escolhidos, permitindo que o cliente continue navegando antes de finalizar seu pedido. Além disso, o sistema deverá possibilitar:

- Alteração da quantidade de itens;
 - Remoção de produtos;
 - Atualização automática do valor total da compra.
-

RF05 – Finalizar Pedido via WhatsApp

Após selecionar os produtos desejados, o usuário poderá finalizar o pedido através da integração com o WhatsApp.

O sistema deverá gerar automaticamente uma mensagem contendo:

- Produtos selecionados;
- Quantidade;
- Valor total;
- Dados informados pelo cliente.

Essa abordagem elimina a necessidade de um sistema complexo de pagamentos e aproveita um canal de comunicação amplamente utilizado pelos clientes da empresa.

RF06 – Gerenciar Conteúdo do Sistema

O administrador deverá possuir uma área exclusiva para gerenciamento das informações do sistema.

Entre as operações permitidas estão:

- Cadastrar produtos;
- Editar produtos;
- Excluir produtos;
- Gerenciar categorias;
- Atualizar informações institucionais;
- Gerenciar dados gerais da plataforma.

Esse requisito garante que o conteúdo permaneça sempre atualizado sem necessidade de alterações diretamente no código.

RF07 – Cadastro Opcional de Clientes (Leads)

O sistema poderá registrar informações básicas dos clientes de forma opcional.

Esses dados poderão ser utilizados para:

- Histórico de contatos;
- Relacionamento com clientes;
- Envio de promoções;
- Newsletter;
- Campanhas de marketing.

Como o cadastro é opcional, o processo de compra permanece simples e rápido.

RF08 – Página para Revendedores

O sistema deverá disponibilizar uma página específica destinada aos revendedores.

Essa área permitirá que interessados em comercializar os produtos da empresa possam:

- Conhecer o programa de revenda;
- Entrar em contato com o setor comercial;
- Solicitar informações sobre parcerias.

Esse recurso amplia as oportunidades comerciais da empresa.

RF09 – Informações das Lojas

O sistema deverá apresentar informações das lojas físicas da empresa.

Entre os dados exibidos estão:

- Endereço;
- Telefone;
- Horário de funcionamento;
- Localização.

Essas informações facilitam o contato dos clientes e fortalecem a presença da empresa nos canais digitais.

RF10 – Informações Institucionais

O sistema deverá disponibilizar informações sobre a empresa, incluindo:

- História;
- Missão;
- Visão;
- Valores;
- Contatos.

Essa seção fortalece a identidade institucional e transmite maior credibilidade aos clientes.

RF11 – Controle de Pedidos

O sistema deverá registrar todos os pedidos realizados pelos clientes.

O controle permitirá armazenar informações como:

- Dados do cliente;
- Produtos solicitados;
- Quantidades;
- Data do pedido;
- Status;
- Atualização do estoque.

Essa funcionalidade auxilia na organização interna e no acompanhamento das vendas.

2.2 Requisitos Não Funcionais

Os requisitos não funcionais definem padrões de qualidade do sistema, garantindo desempenho, segurança, disponibilidade e boa experiência para os usuários.

RNF01 – Compatibilidade Responsiva

O sistema deverá adaptar automaticamente sua interface para diferentes dispositivos, incluindo:

- Computadores;
- Tablets;
- Smartphones.

A responsividade garante que todas as funcionalidades permaneçam acessíveis independentemente do tamanho da tela, proporcionando uma experiência consistente ao usuário.

RNF02 – Tempo de Carregamento

As páginas deverão apresentar carregamento rápido, minimizando o tempo de espera do usuário.

Um bom desempenho reduz abandonos, melhora a experiência de navegação e contribui para melhores resultados em mecanismos de busca.

RNF03 – Segurança de Acesso Administrativo

As funcionalidades administrativas deverão estar protegidas por mecanismos de autenticação e autorização.

O sistema deverá garantir que apenas usuários autorizados possam:

- Cadastrar produtos;
- Editar informações;
- Excluir registros;
- Alterar configurações do sistema.

Esse requisito protege dados sensíveis e reduz riscos de acessos indevidos.

RNF04 – Disponibilidade do Sistema

O sistema deverá permanecer disponível durante a maior parte do tempo, reduzindo interrupções e falhas de funcionamento.

A alta disponibilidade garante que clientes consigam consultar produtos e realizar pedidos sempre que necessário, aumentando a confiabilidade da aplicação.

RNF05 – Facilidade de Navegação

A interface deverá ser simples, intuitiva e organizada.

Os usuários deverão conseguir localizar informações e executar tarefas com poucos cliques, reduzindo erros e aumentando a satisfação durante a utilização do sistema.

Aspectos considerados incluem:

- Menus organizados;
 - Layout limpo;
 - Navegação intuitiva;
 - Ícones compreensíveis;
 - Fluxo consistente entre páginas.
-

3. Tecnologias e Dependências

Esta seção lista as decisões tecnológicas e as motivações para cada escolha. Onde houver alternativas relevantes, comentamos os motivos para a seleção atual.

3.1 Backend (**tny-backend**)

Tecnologia	Versão	Papel	Justificativa
Node.js	≥ 22	Runtime JavaScript	Versão LTS moderna com melhorias de performance e suporte a recursos recentes do ECMAScript.
TypeScript	^6.0.3	Tipagem estática	Fornece segurança de tipos, melhor autocompletar e refatoração mais segura.
Fastify	^5.8.5	Framework HTTP de alta performance	Alta performance e ecossistema de plugins; integra fácil com testes via <code>app.inject()</code> .
Prisma	^7.8.0	ORM	Cliente tipado, migrations e facilidade de suporte a múltiplos providers.
Zod	^4.4.3	Validação	Validação compartilhada entre runtime e tipos, reduz inconsistências.
<code>@fastify/jwt</code>	^10.1.0	Autenticação JWT	Integração direta com Fastify; simples e suficiente para autenticação admin.
<code>@fastify/swagger / UI</code>	^9.x / ^5.x	OpenAPI / UI	Facilita documentação interativa para desenvolvedores e testes manuais.
<code>@fastify/cors</code>	^11.2.0	CORS	Controle de origem para segurança no frontend.
bcryptjs	^3.0.3	Hash de senhas	Biblioteca leve e compatível com ambientes sem builds nativos.
dayjs	^1.11.21	Datas	Biblioteca pequena e eficiente para manipulação de datas.
pg	^8.21.0	Driver PostgreSQL	Necessário em produção com Postgres.
Vitest	^4.1.9	Testes	Rápido e compatível com TS/Vite.
tsup / tsx	^8.5.1 / ^4.22.4	Bundling/Dev	<code>tsup</code> para builds e <code>tsx</code> para dev runtime.
ESLint + Prettier	^10.4.1 / ^3.8.4	Lint/format	Consistência e qualidade de código.

Banco de dados:

- **Desenvolvimento/Testes:** SQLite — zero-config e rápido em CI.
- **Produção/Docker:** PostgreSQL 16 — robustez e recursos de produção.

3.2 Frontend (**tny-frontend**)

Tecnologia	Versão	Papel	Justificativa
Node.js	≥ 22	Runtime para build	Consistência entre backend e frontend.
TypeScript	~6.0.2	Tipagem estática	Segurança de tipos no cliente.
React	^19.2.7	Biblioteca de UI	Ecossistema maduro e produtividade.

Tecnologia	Versão	Papel	Justificativa
React Router DOM	^7.18.1	Roteamento	Padrão para SPAs React.
Vite	^8.1.1	Build/dev	Inicialização rápida e HMR.
Tailwind CSS	^4.3.2	Estilização	Estilização utilitária para acelerar builds de UI.
Lucide React	^1.22.0	Ícones SVG	Conjunto leve e consistente.

Decisão de UI: React + Tailwind prioriza rapidez de desenvolvimento, facilidade de manutenção e baixo acoplamento a design systems proprietários.

4. Pré-requisitos

Antes de iniciar, garanta que os seguintes softwares estão instalados. As versões mínimas refletem compatibilidade com as ferramentas do projeto.

Software	Versão mínima	Verificação
Node.js	22.x	<code>node --version</code>
npm	10.x	<code>npm --version</code>
Git	qualquer	<code>git --version</code>
Docker	24.x (opcional)	<code>docker --version</code>
Docker Compose	2.x (opcional)	<code>docker compose version</code>

Observações:

- Docker é opcional em dev, recomendado para reproduzir ambiente de produção (Postgres).
- No Windows, atente para permissões de volume Docker e caminhos ao usar SQLite.

5. Instalação e Configuração

Esta seção descreve passos práticos para rodar o projeto localmente e também as razões por trás de passos críticos (migrations, variáveis sensíveis, seeds).

5.1 Clonar o Repositório

```
# Clone o repositório do backend e frontend (ou baixe o monorepo conforme disponível)
git clone https://github.com/pi-tny/tny-backend
git clone https://github.com/pi-tny/tny-frontend
```

5.2 Configurar e Instalar o Backend

Passos essenciais e justificativas:

```
cd tny-backend

# 1. Copiar arquivo de variáveis de ambiente
```



```
cp .env.example .env

# 2. Editar o .env com suas configurações (ver seção 4.4)
#   - JWT_SECRET é obrigatório para segurança dos tokens
#   - DATABASE_URL deve apontar para SQLite em dev ou Postgres em produção

# 3. Instalar dependências
npm install

# 4. Aplicar as migrations do banco de dados
npm run db:migrate

# 5. (Opcional) Popular com dados de exemplo para facilitar testes locais
npm run db:seed
```

Por que aplicar migrations localmente?

- As migrations garantem que o schema do banco esteja sincronizado com o código; evitam erros sutis em runtime e garantem que os testes de integração executem em um schema conhecido.

5.3 Configurar e Instalar o Frontend

```
cd tny-frontend

# Instalar dependências
npm install
```

O frontend atualmente não exige variáveis de ambiente obrigatórias. Se você for integrar APIs privadas, defina `VITE_API_URL` conforme necessário.

5.4 Variáveis de Ambiente do Backend

O arquivo `.env` (criado a partir de `.env.example`) controla o comportamento da API. Algumas variáveis são sensíveis e não devem ser commitadas.

Variável	Obrigatória	Padrão	Descrição	Observações
<code>DATABASE_URL</code>	Sim	—	URL de conexão com o banco. Ex: <code>file:./prisma/dev.db</code> (SQLite) ou <code>postgresql://user:pass@host:5432/db</code> (Postgres)	Use SQLite para dev/test por conveniência; Postgres em produção.
<code>JWT_SECRET</code>	Sim	—	Chave secreta para assinar e verificar tokens JWT.	Use um segredo forte em produção; rotacione se houver suspeita de comprometimento.
<code>DATABASE_PROVIDER</code>	Não	<code>sqlite</code>	Provedor do banco: <code>sqlite</code> ou <code>postgres</code> .	Facilita scripts que escolhem

Variável	Obrigatória	Padrão	Descrição	Observações
				migrations/mapping no build.
PORT	Não	3000	Porta em que a API vai escutar	Alterar se conflito de portas.
NODE_ENV	Não	dev	Ambiente: dev, test ou production	Controla logs e otimizações.
CORS_ORIGIN	Não	*	Origem permitida para CORS.	Em produção, restrinja ao domínio do frontend.
WHATSAPP_NUMBER	Não	""	Número do WhatsApp da loja (formato: 5585981000000).	Usado na geração do link de checkout; opção segura para testes locais.

Exemplo mínimo para desenvolvimento:

```
DATABASE_PROVIDER="sqlite"
DATABASE_URL="file:./prisma/dev.db"
PORT=3000
NODE_ENV=dev
JWT_SECRET="minha-chave-secreta-muito-longa-e-segura"
CORS_ORIGIN="http://localhost:5173"
WHATSAPP_NUMBER="5585981025616"
```

Segurança: nunca comite chaves reais no repositório. Utilize variáveis de ambiente no CI/CD e mecanismos de secret management em produção.

6. Execução

Guia rápido para executar o sistema em desenvolvimento e produção, com observações sobre quando usar cada modo.

6.1 Modo Desenvolvimento (sem Docker)

Execute backend e frontend em terminais separados para desenvolvimento iterativo:

Terminal 1 — Backend:

```
cd tny-backend
npm run dev
```

Endpoints locais úteis:

- API: <http://localhost:3000>
- Swagger UI: <http://localhost:3000/docs>

- Health check: <http://localhost:3000/health>

Terminal 2 — Frontend:

```
cd tny-frontend
npm run dev
```

Frontend local: <http://localhost:5173>

Quando usar este modo: rápido para desenvolvimento, hot-reload e depuração. Não reproduz o ambiente de produção quando o banco é Postgres.

6.2 Modo Produção com Docker

Para um ambiente mais próximo do real (Postgres + API), utilize Docker Compose:

```
cd tny-backend

# 1. Copiar e editar .env conforme necessário
cp .env.example .env

# 2. Subir os containers (build + start)
docker compose up --build

# 3. (Opcional) Popular o banco após subirdocker compose exec api npm run db:seed
```

Por que usar Docker em produção?

- Reproduz dependências (Postgres) de forma previsível e isolada, facilita deploy e CI.

Parar containers:

```
docker compose down
```

Remover volumes (apaga dados):

```
docker compose down -v
```

6.3 Build Manual para Produção (sem Docker)

```
# Backend
cd tny-backend
npm run build
npm start

# Frontend
cd tny-frontend
```

```
npm run build
npm run preview
```

Use o build manual para deploys sem containerização (ex.: servidores tradicionais). Em ambientes serverless prefira `npm run build:vercel` conforme a configuração do projeto.

7. Estrutura do Projeto

O repositório é organizado para separar claramente responsabilidades e facilitar testes, manutenção e deploy.

7.1 Visão Geral da Raiz

```
tny/
├── tny-backend/    ← API REST (Fastify + Prisma + TypeScript)
└── tny-frontend/  ← SPA (React + Vite + Tailwind)
```

NOTA: manter backend e frontend separados facilita deploys independentes e pequenos times trabalhando em paralelo.

7.2 `tny-backend/`

Estrutura pensada em camadas (HTTP → Controller → Use Case → Repository) para isolar lógica de negócio e permitir testes unitários com repositórios in-memory.

Principais diretórios e papéis:

- `src/app.ts` / `src/server.ts`: inicialização e boot da aplicação.
- `src/env`: validação de variáveis de ambiente com Zod para evitar configuração inválida em runtime.
- `src/http`: controllers, middlewares e mapeamentos HTTP — ponto de integração entre transporte (Fastify) e lógica de aplicação.
- `src/use-cases`: lógica de negócio pura, sem dependências de infraestrutura.
- `src/repositories`: contratos e implementações (Prisma e in-memory) — facilita mocks para testes.
- `prisma/`: schema, migrations e seed — centraliza regras do modelo de dados.

Essa separação ajuda a manter as regras de negócio testáveis e independentes da tecnologia de persistência.

```
tny-backend/
├── src/
│   ├── server.ts    ← Código-fonte principal
│   │               ← Entry point: chama app.listen()
│   └── app.ts       ← Instância Fastify + plugins + rotas (sem listen)
│   ├── env/
│   │   └── index.ts ← Validação de variáveis de ambiente com Zod
│   ├── @types/
│   │   └── fastify-jwt.d.ts ← Augmentação de tipos: payload JWT { sub: number }
│   ├── lib/
│   │   └── prisma.ts ← Singleton do PrismaClient (adapter SQLite ou
│   │               Postgres)
```

http/	
error-handler.ts	← Handler global de erros do Fastify
http-schemas.ts	← Schemas Zod compartilhados (erro, paginação, envelope)
map-domain-error.ts	← Mapeia erros de domínio → resposta HTTP
openapi.ts	← Metadados OpenAPI + helper bearerSecurity
openapi-schemas.ts	← Registro de schemas nomeados para \$ref no Swagger
middlewares/	
verify-jwt.ts	← Guard de autenticação Bearer JWT
controllers/	← Um subdiretório por domínio
health/	← GET /health
admin-auth/	← Login, me, logout
categories/	← Listagem pública
admin-categories/	← CRUD admin
products/	← Listagem pública + detalhe + relacionados
admin-products/	← CRUD admin + set-categories
admin-variants/	← CRUD variantes
admin-images/	← CRUD imagens
orders/	← Criar pedido (público)
admin-orders/	← Listagem + status admin
leads/	← Cadastro de lead (público)
admin-leads/	← Listagem + exclusão admin
admin-admins/	← CRUD de administradores
use-cases/	← Lógica de negócio pura (sem Prisma, sem HTTP)
categories/	← 5 use cases
products/	← 7 use cases
variants/	← 4 use cases
images/	← 3 use cases
orders/	← 4 use cases
leads/	← 3 use cases
admins/	← 4 use cases
auth/	← 2 use cases (login, me)
errors/	← Classes de erro de domínio
factories/	← Injeção de dependência manual (make-*-use-case.ts)
repositories/	← Interfaces e implementações dos repositórios
*-repository.ts	← Interfaces (contrato mínimo por domínio)
prisma/	← Implementações com Prisma
in-memory/	← Implementações fake para testes unitários
utils/	
prisma-search.ts	← insensitiveContains (portável SQLite/Postgres)
product-view.ts	← resolveFinalPrice + resolveCoverImage
whatsapp.ts	← Geração de link wa.me com mensagem do pedido
test/	← Helpers de teste (reset DB, criar e autenticar)
prisma/	
schema.prisma	← Schema Prisma (modelos + datasource)
seed.ts	← Seed: cria admin, categorias, produtos, pedidos,
leads	
dev.db	← SQLite de desenvolvimento (ignorado no git)
migrations/	
sqlite/	← Migrations para SQLite

```

├── postgres/                ← Migrations para PostgreSQL
├── api/
│   └── index.js             ← Entry point serverless para Vercel
├── scripts/
│   ├── prisma.mjs           ← Wrapper: reescreve provider no schema antes de rodar
├── Prisma CLI
│   └── seed-pg.mjs          ← Gera client Postgres, migra e popula (dev com
├── Postgres local)
├── test/
│   └── global-setup.ts      ← Vitest: cria e destrói prisma/test.db antes/depois
├── da suíte
├── docs/
│   ├── ROADMAP.md           ← Status de implementação por domínio
│   └── SKILL-*.md           ← Guias de arquitetura interna
├── public/
│   └── index.html           ← Página estática de fallback
├── Dockerfile
├── docker-compose.yml
├── vercel.json
├── tsconfig.json
├── tsup.config.ts
├── vitest.config.mts
├── prisma.config.ts
├── eslint.config.js
├── .prettierrc
├── .env.example
└── package.json

```

7.3 tny-frontend/

Organizado para ser uma SPA leve e extensível:

- **src/context**: providers e hooks para estado global (carrinho, toasts).
- **src/components** e **src/pages**: componentes reutilizáveis e views.
- **Vite + Tailwind**: escolha que prioriza velocidade de desenvolvimento e tamanho reduzido do bundle.

Design decision: o frontend assume que o backend provê endpoints REST bem documentados; a troca para outro cliente ou mobile app é direta quando a API tem contratos estáveis.

```

tny-frontend/
├── src/
│   ├── main.tsx             ← Entry point React: monta #root com StrictMode
│   ├── App.tsx              ← Root: BrowserRouter + ToastProvider +
├── CarrinhoProvider
│   ├── index.css            ← @import "tailwindcss"
│   └── types/
│       └── index.ts         ← Interfaces: Produto e CartItem

```

data/	
└─ produtos.ts	← Array estático de produtos da vitrine
context/	
└─ CarrinhoContext.tsx	← Estado global do carrinho + persistência
localStorage	
└─ ToastContext.tsx	← Notificações toast com auto-dismiss
└─ useCarrinho.ts	← Hook: useCarrinho()
└─ useToast.ts	← Hook: useToast()
routes/	
└─ AppRoutes.tsx	← Definição de todas as rotas da SPA
components/	
└─ Header.tsx	← Navegação principal + ícone do carrinho
└─ Footer.tsx	← Rodapé com contatos e newsletter
└─ CardProduto.tsx	← Card de produto reutilizável
└─ Toast.tsx	← Overlay de notificação
└─ NewsletterForm.tsx	← Formulário de inscrição
└─ Promocoes.tsx	← Componente/página de promoções
pages/	
└─ Home.tsx	← Vitrine principal com filtros de categoria
└─ Produto.tsx	← Detalhe do produto: galeria + seleção de variante
└─ Carrinho.tsx	← Carrinho com controles de quantidade
└─ Checkout.tsx	← Resumo do pedido + botão WhatsApp
└─ Institucional.tsx	← Página "Quem Somos"
└─ Revendedor.tsx	← Formulário de contato para revendedores
└─ admin/	
└─ Login.tsx	← Login do painel admin
└─ Dashboard.tsx	← Pannel com métricas básicas
└─ CadastroProduto.tsx	← Formulário de cadastro de produto
└─ GerenciarEstoque.tsx	← Tabela de produtos com exclusão
assets/	← Imagens e logos da marca
public/	
└─ favicon.svg	
└─ icons.svg	
index.html	← Shell HTML da SPA
vite.config.ts	
tailwind.config.js	
postcss.config.js	
tsconfig.json	
package.json	

8. Arquitetura do Backend

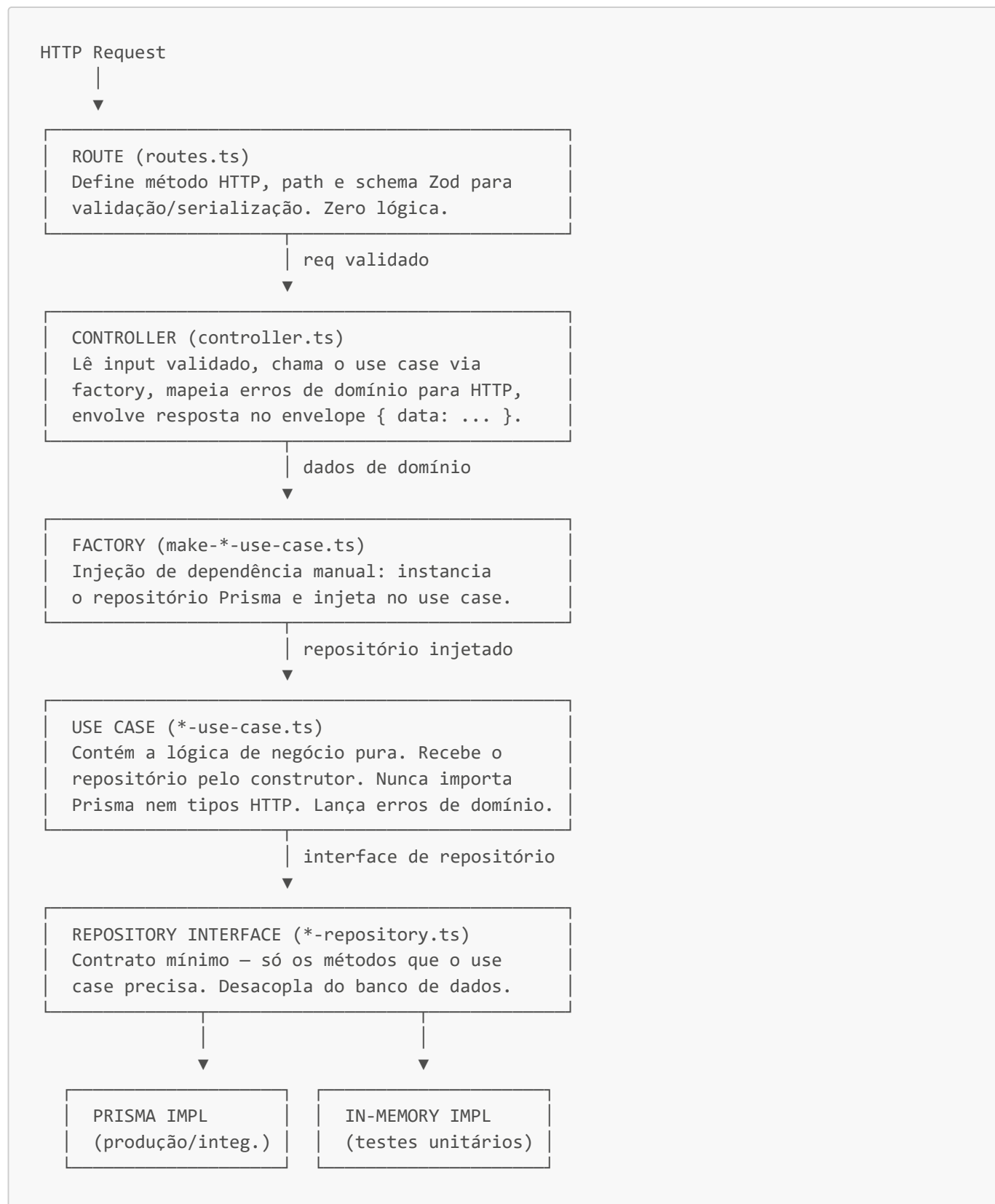
O backend segue um padrão em camadas pensado para clareza, testabilidade e facilidade de manutenção. A ideia central é manter a lógica de negócio (**use-cases**) isolada de detalhes de transporte (HTTP) e persistência (Prisma).

8.1 Padrão em Camadas

Fluxo simplificado:

HTTP Request → Controller → Use Case → Repository → Database

Fluxo Detalhado:



Principais vantagens:

- Testabilidade: **use-cases** são funções/padrões testáveis sem infra.
- Substituibilidade: trocar Prisma por outro adaptador requer mudanças localizadas nos repositórios.
- Clareza: controllers tratam mapeamento HTTP, não regras de negócio.

8.2 Envelopes de Resposta

Adotamos um envelope uniforme para facilitar consumo por clientes e reduzir ambiguidade nos contratos:

- Sucesso: `{ "data": ... }`
- Páginação: `{ "data": [...], "meta": { "page", "limit", "total", "total_pages" } }`
- Erros de domínio e validação seguem estrutura `{ "error": { "code": "...", "message": "..." } }`.

NOTA: envelopes consistentes tornam parsing de resposta mais previsível em clientes e reduzem código de tratamento de erro espalhado.

8.3 Autenticação

- Método: **Bearer JWT** (cabeçalho `Authorization: Bearer <token>`)
- Biblioteca: `@fastify/jwt`
- Payload: `{ sub: number }` (ID do administrador)
- Expiração: `1d` (um dia)
- Rotas públicas: nenhum token necessário
- Rotas admin: protegidas pelo middleware `verifyJwt` (retorna `401 UNAUTHORIZED` se ausente/inválido)
- Método: **Bearer JWT** com payload reduzido (`{ sub: number }`).
- Justificativa: JWT é suficiente para autenticação stateless de administradores; simplifica escalabilidade e remove a necessidade de sessão server-side.
- Expiração padrão: `1d`. Para ambientes mais sensíveis, considerar refresh tokens ou sessão curta.

8.4 Mapa de Erros de Domínio

Mapear erros de domínio para códigos HTTP e chaves JSON ajuda clientes a automatizar tratamentos (re-tentativa, mensagens ao usuário, logs):

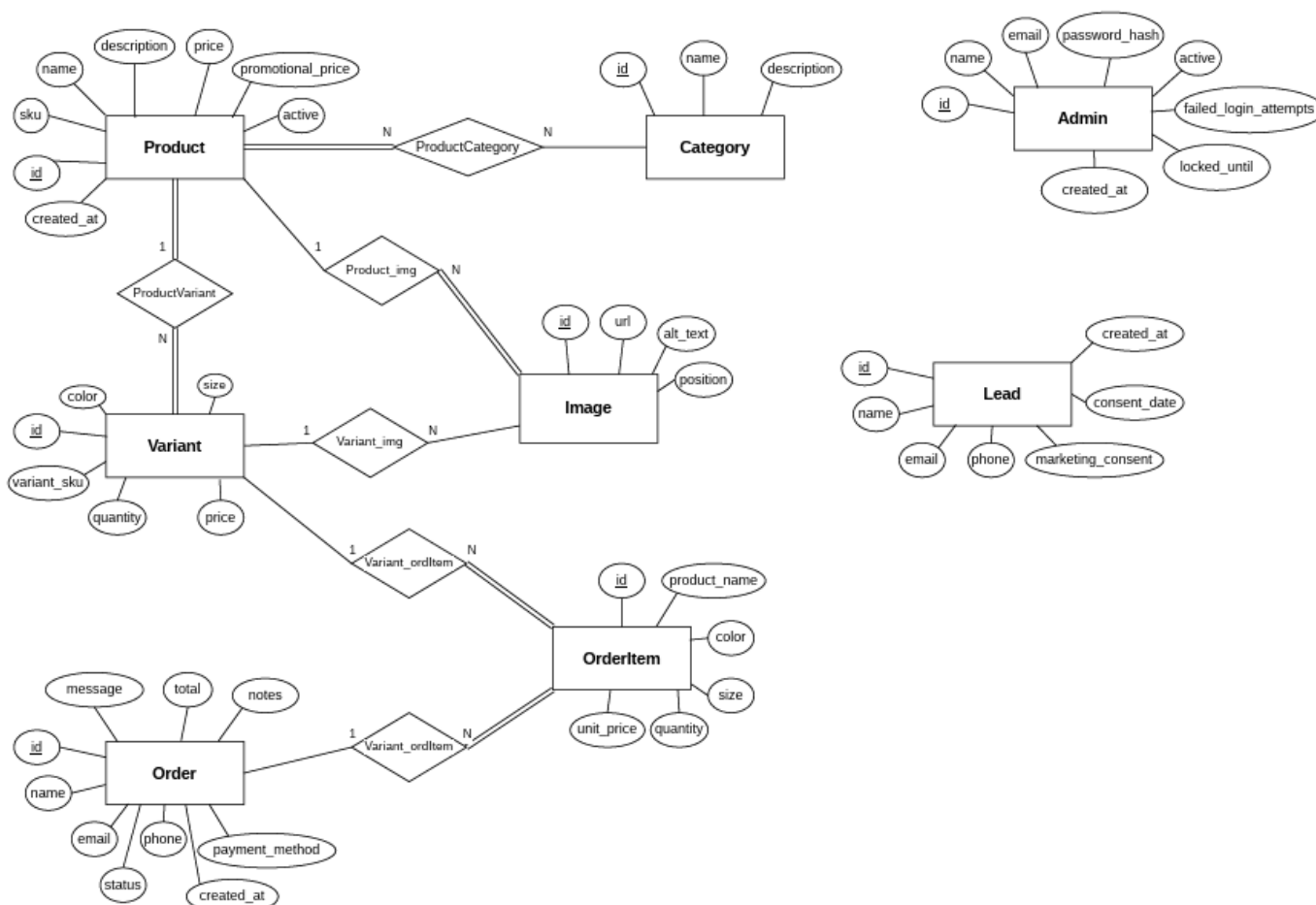
Erro de Domínio	HTTP	Código JSON
<code>ResourceNotFoundError</code>	404	<code>RESOURCE_NOT_FOUND</code>
<code>InvalidCredentialsError</code>	401	<code>INVALID_CREDENTIALS</code>
<code>ProductSkuAlreadyExistsError</code>	409	<code>PRODUCT_SKU_ALREADY_EXISTS</code>
<code>VariantSkuAlreadyExistsError</code>	409	<code>VARIANT_SKU_ALREADY_EXISTS</code>
<code>AdminAlreadyExistsError</code>	409	<code>ADMIN_ALREADY_EXISTS</code>
<code>InvalidPromotionalPriceError</code>	422	<code>INVALID_PROMOTIONAL_PRICE</code>
Erro de validação Zod/Fastify	400	<code>VALIDATION_ERROR</code>
Erro inesperado	500	<code>INTERNAL_SERVER_ERROR</code>

Explicação: usar códigos e mensagens consistentes facilita instrumentação (logs/telemetria) e testes automatizados de fluxos de erro.

9. Banco de Dados

O projeto usa Prisma como camada de acesso e abstração do banco de dados. A estratégia é suportar dois providers (SQLite para dev/test e Postgres para produção).

9.1 Modelo e ER (resumo)



O diagrama ER centraliza entidades: **Product**, **Variant**, **Image**, **Category**, **Order**, **OrderItem**, **Lead**, **Admin**.

Decisões importantes:

- Normalização moderada: campos denormalizados em **OrderItem** (nome, cor, tamanho, unit_price) garantem histórico imutável de pedidos.
- Indexação: campos usados em busca e filtros (nome, sku, category_id) são indexados para performance.

9.2 Regras de Negócio do Banco

- Preço final do produto: se **promotional_price** não for nulo → usa o promocional; senão, se a variante tiver **price** → usa o da variante; senão → usa o **price** do produto.
- Imagem de capa: ordena por **position**, prefere imagens sem **variant_id** (imagens gerais); fallback para qualquer imagem disponível. Isso garante que o produto sempre tenha uma imagem de apresentação.
- Pedidos: campos como **product_name**, **color**, **size** e **unit_price** em **OrderItem** são "congelados" no momento da criação do pedido para preservar histórico comercial.

- Status do pedido: **new**, **confirmed**, **shipped**, **delivered**, **cancelled** — validados pela camada Zod. Usamos validação na aplicação para permanecer independente de limitações do BD (especialmente em SQLite).

Motivação: a combinação de Prisma + migrations e seed scripts facilita a transição entre providers e mantém a integridade dos dados através de constraints e validações na aplicação.

10. API — Referência de Rotas

As rotas seguem convenções REST simples e consistentes. A documentação interativa (</docs>) fornece exemplos de request/response e os schemas Zod correspondentes.

Rotas Públicas

Método	Caminho	Descrição	Observação
GET	/health	Health check — retorna status da API	Útil para readiness/monitoramento
GET	/categories	Lista todas as categorias	Paginada/filtrável conforme necessidade
GET	/categories/:id	Detalhe de uma categoria	Inclui counts/opcionais de produtos
GET	/products	Lista produtos (filtro por categoria, busca, paginação)	Busca textual usa prisma-search para compatibilidade SQLite/Postgres
GET	/products/:id	Detalhe de um produto com variantes e imagens	Responde com visão pronta para vitrine (preço final, imagem de capa)
GET	/products/:id/related	Lista produtos relacionados (mesmas categorias)	Algoritmo simples por interseção de categorias
POST	/orders	Cria um pedido (retorna link WhatsApp)	Persiste Order + OrderItems e devolve link para WhatsApp com mensagem formatada
POST	/leads	Cadastra lead para newsletter	Registro simplificado com validação de e-mail
POST	/admin/auth/login	Autenticação — retorna JWT	Endpoint público para obter token admin

Rotas Administrativas (requer Bearer JWT)

As rotas sob </admin> exigem autenticação e atendem operações de gestão:

Método	Caminho	Descrição
GET	/admin/auth/me	Retorna dados do admin autenticado
POST	/admin/auth/logout	Logout (invalidação client-side)
GET	/admin/categories	Lista categorias (admin)
POST	/admin/categories	Cria categoria

Método	Caminho	Descrição
PUT	/admin/categories/:id	Atualiza categoria
DELETE	/admin/categories/:id	Remove categoria
GET	/admin/products	Lista produtos (admin, com inativos)
POST	/admin/products	Cria produto
GET	/admin/products/:id	Detalhe do produto (admin)
PUT	/admin/products/:id	Atualiza produto
DELETE	/admin/products/:id	Inativa produto (soft delete)
PUT	/admin/products/:id/categories	Define categorias do produto
GET	/admin/products/:id/variants	Lista variantes de um produto
POST	/admin/products/:id/variants	Cria variante
PUT	/admin/variants/:id	Atualiza variante
DELETE	/admin/variants/:id	Remove variante
POST	/admin/products/:id/images	Adiciona imagem ao produto
PUT	/admin/images/:id	Atualiza dados da imagem
DELETE	/admin/images/:id	Remove imagem
GET	/admin/orders	Lista pedidos (paginado, filtro por status)
GET	/admin/orders/:id	Detalhe do pedido com itens
PATCH	/admin/orders/:id/status	Atualiza status do pedido
GET	/admin/leads	Lista leads (paginado)
DELETE	/admin/leads/:id	Remove lead
GET	/admin/admins	Lista administradores
POST	/admin/admins	Cria administrador
PUT	/admin/admins/:id	Atualiza administrador
DELETE	/admin/admins/:id	Remove administrador

Documentação interativa: acesse <http://localhost:3000/docs> com a API rodando para explorar schemas e exemplos.

11. Testes

A suíte de testes foi desenhada para garantir qualidade em diferentes níveis: unitário (veloz) e integração (mais completo).

11.1 Estratégia

Nível	Localização	Ferramentas	Banco	Objetivo
Unitário	<code>src/use-cases/**/*.spec.ts</code>	Vitest + repositórios in-memory	Nenhum (memória)	Validar regras de negócio isoladas
Integração	<code>src/http/controllers/**/*.spec.ts</code>	Vitest + <code>app.inject()</code>	SQLite (<code>prisma/test.db</code>)	Testar pipeline HTTP completo sem servidor real

Motivação:

- Testes unitários são rápidos e fornecem cobertura para lógica crítica.
- Testes de integração garantem que controllers, validações e migrations funcionem juntos.

No Front-end, foram realizados teste manuais durante integração e pós integração para checagem de possíveis erros e seus tratamentos.

11.2 Executar os Testes

```
cd tny-backend

# Executar todos os testes
npm test

# Executar em modo watch
npm run test:watch

# Executar com relatório de cobertura
npm run test:coverage
```

11.3 Cobertura Mínima

Thresholds aplicados sobre `src/use-cases/**`:

Métrica	Threshold
Lines	≥ 90%
Functions	≥ 90%
Branches	≥ 90%
Statements	≥ 90%

Esses limites refletem a prioridade em garantir qualidade da lógica de domínio.

11.4 Isolamento dos Testes

- O banco de testes (`prisma/test.db`) é criado no `test/global-setup.ts` e removido no teardown para evitar poluição entre execuções.

- Cada teste de integração chama `resetDatabase()` no `beforeEach` para garantir estado limpo.
- Os testes rodam sequencialmente (`fileParallelism: false`) para evitar condições de corrida comuns com SQLite.

Observação: ao migrar para Postgres no CI, ajustar configurações para paralelismo seguro e uso de bancos isolados por job.

12. Containerização e Deploy

O projeto suporta deploy em containers (Docker Compose) e deployment serverless (Vercel). A decisão por containerização visa reproduzibilidade local e produção consistente.

12.1 Estado Atual — Backend Containerizado

O backend e o banco PostgreSQL são orquestrados via `docker-compose.yml` para ambientes de produção/integração.

Pontos relevantes:

- `db` (Postgres) separado do `api` para isolar dados e permitir escalabilidade.
- Healthchecks para dependência de banco e `depends_on` com condição de saúde para evitar race conditions no boot.
- O `Dockerfile` usa etapas de build que geram o Prisma Client e constroem o bundle de produção.

Vantagens do fluxo Docker:

- Reprodutibilidade entre desenvolvedores e entre CI/CD.
- Isolamento de dependências (Postgres) sem necessidade de instalar DB local.

12.2 Containerização do Frontend

Plano: build multi-stage com `node` (builder) e `nginx` (runner) para imagem final enxuta.

Justificativa: reduz tamanho da imagem e converte o output do Vite em arquivo estático servido por Nginx, prática comum para SPAs.

12.3 Arquitetura alvo no `docker-compose`

Objetivo final: 3 serviços na mesma rede (`db`, `api`, `frontend`), com variáveis de ambiente seguras e volumes para persistência de dados.

12.4 Deploy Serverless — Vercel (Backend)

Opcionalmente, o backend pode ser empacotado para Vercel (serverless), útil para deploys rápidos sem gerenciar infraestrutura. Quando deployado serverless, é importante garantir que o provider do banco (Postgres) seja acessível e que o Prisma Client seja gerado para o target (Postgres) durante o build.

Trade-offs:

- Serverless: escalabilidade automática e custo variável; cold starts e limites de execução podem impactar workloads pesadas.
 - Containers: maior controle, fácil replicação de ambiente, overhead operacional.
-

13. Scripts — Referência Rápida

Backend (`tny-backend`)

Comando	Descrição
<code>npm run dev</code>	Inicia servidor em modo watch (tsx) — hot reload
<code>npm run build</code>	Compila para <code>dist/</code> com tsup
<code>npm run build:vercel</code>	Build para deploy Vercel (gera client Postgres antes)
<code>npm start</code>	Inicia o build compilado (<code>dist/server.js</code>)
<code>npm run lint</code>	Verifica erros de lint (ESLint)
<code>npm run lint:fix</code>	Corrige erros de lint automaticamente
<code>npm run format</code>	Formata código com Prettier
<code>npm test</code>	Executa a suíte de testes
<code>npm run test:watch</code>	Testes em modo watch
<code>npm run test:coverage</code>	Testes com relatório de cobertura
<code>npm run db:migrate</code>	Aplica migrations pendentes (cria tabelas)
<code>npm run db:generate</code>	Regenera o Prisma Client após mudanças no schema
<code>npm run db:seed</code>	Popula o banco com dados de exemplo
<code>npm run db:seed:pg</code>	Popula um banco PostgreSQL local (para desenvolvimento com Postgres)
<code>npm run db:reset</code>	Apaga e recria o banco (destrói dados — use com cuidado)
<code>npm run db:studio</code>	Abre o Prisma Studio (GUI para o banco)

Frontend (`tny-frontend`)

Comando	Descrição
<code>npm run dev</code>	Inicia servidor de desenvolvimento Vite
<code>npm run build</code>	Gera build de produção em <code>dist/</code>
<code>npm run preview</code>	Serve o build de produção localmente
<code>npm run lint</code>	Verifica erros de lint (ESLint)

Esses scripts cobrem fluxo de desenvolvimento, build, testes e operações básicas de banco.